

# JavaScript Functions - Complete Guide

---

## What is a Function?

- Function is object.
  - function is a block of instruction which is used to perform a specific task.
  - A function is executed only when it is called.
  - The main advantages of function is we can achieve code reusability.
  - To call a function we can achieve code reusability.
  - To call a function we need its reference and ().
  - Name of a function is variable which holds the reference of function object.
  - creating a function using 'function' keyword supports function hoisting.
  - Therefore we can also call a function before function declaration.
  - when we try to log function name the entire function definition is printed.
  - The scope within function block is known as local scope.
  - variable written inside function even using var have local scope.
  - In javascript 'this' is a property of every function. [every function will have 'this' keyword except arrow function]
  - Any member with A **function** is a fundamental building block in JavaScript that serves multiple purposes:
1. **Block of Instructions:** A function is a reusable block of code designed to perform a specific task
  2. **Object Nature:** In JavaScript, functions are first-class objects, meaning they can be:
    - Stored in variables
    - Passed as arguments to other functions
    - Returned from other functions
    - Have properties and methods
  3. **Execution Control:** A function executes only when it is called/invoked
  4. **Code Reusability:** The main advantage is achieving code reusability and modularity

## Function Anatomy

```
function functionName(parameters) {  
    // Function body - code to be executed  
    return value; // Optional return statement  
}  
  
// Function call  
functionName(arguments);
```

## Key Terms:

- **Parameters:** Variables listed in function definition (`a`, `b`)
- **Arguments:** Actual values passed when calling function (`2`, `3`)
- **Function Name:** Variable that holds the reference to function object
- **Return Value:** Every function returns something (undefined if no explicit return)

## Function Hoisting

Functions created with the `function` keyword support **hoisting**, meaning you can call them before their declaration:

```
// This works due to hoisting
console.log("Start");
greet("Avinash"); // Called before declaration

function greet(user) {
  console.log(`Namaste ${user}...!!`);
}
console.log("End");

// Output:
// Start
// Namaste Avinash...!!
// End
```

## Types of Functions

### 1. Function Declaration

```
function greet(user) {
  console.log(`Hello ${user}`);
}

greet("Avinash");
```

#### Characteristics:

- Supports hoisting
- Function name is mandatory
- Can be called before declaration

## 2. Function Expression (First-Class Function)

```
const greet = function(user) {  
  console.log("Username:", user);  
  return 10;  
};  
  
greet("Avinash");
```

### Characteristics:

- Function is assigned to a variable
- No hoisting (variable hoisting rules apply)
- Function name is optional (anonymous function)
- Cannot be called before declaration

## 3. Immediately Invoked Function Expression (IIFE)

```
// Basic IIFE  
(function greet(user) {  
  console.log(`Namaste ${user}`);  
})("Aditya");  
  
// IIFE with return value  
const res = (function() {  
  console.log("Hahahahahaa...");  
  return "Tu samjha..!!";  
})();  
  
console.log("res:", res); // res: Tu samjha..!!
```

### Characteristics:

- When a function is called as soon as its object is created is known as immediate invoke function.
- The function is not visible(available) outside the scope.
- After grouping it, we have to use parenthesis to call this function.
- Runs only once
- Creates its own scope (prevents global pollution)
- Syntax: `(function() {})( )` - first `()` groups the function, second `()` invokes it

## 4. Arrow Functions (ES6)

```
// Implicit return (single expression)
const greet1 = (user) => console.log(`GM ${user}`);

// Explicit return (block body)
const greet2 = (user) => {
  console.log(`Namaste ${user}`);
  return "Dinga";
};

// No parameters
const sayHello = () => console.log("Hello World");

// Single parameter (parentheses optional)
const square = x => x * x;
```

### Characteristics:

- Introduced in ES6
- Shorter syntax
- If we have only single parameter, It is not necessary to use parenthesis for parameter.
- **Implicit return** :- If there is only one statement and if block is not created then JS engine will return that statement automatically. **Implicit Return Example**
- If there is only **one statement** and the **block ({} ) is not used**, JavaScript will return the result **automatically**.

```
const add = (a, b) => a + b;
console.log(add(2, 3)); // Output: 5
```

- **Explicit return** :- If block is created and function is not returning any value, JS Engine will return undefined. To return a value Explicitly from block, we have to use return keyword. If block is created then we have to use return keyword to return value otherwise JS Engine will return undefined.

### Explicit Return in Arrow Function

- If a block ({} ) is used, JavaScript will return **undefined unless** the **return** keyword is used explicitly.

```
const multiply = (a, b) => {
  return a * b;
};
console.log(multiply(2, 3)); // Output: 6
```

- No `this` binding (inherits from enclosing scope)
- Implicit return for single expressions
- Cannot be used as constructors

## Function Execution Process

When a function is called, JavaScript creates a **Function Execution Context** with two phases:

### Variable Phase (Creation Phase)

```
function sum(a, b) {  
  var result = a + b;  
  console.log("Sum:", result);  
  return result;  
}  
  
const x = sum(10, 20);
```

During variable phase:

1. Memory allocated for parameters (`a, b`) → `undefined`
2. Memory allocated for local variables (`result`) → `undefined`
3. `this` keyword allocated → points to appropriate object
4. Return value allocated → `undefined`

### Execution Phase

1. Parameters get actual values (`a = 10, b = 20`)
2. Code executes line by line
3. Variables get assigned actual values
4. Return statement (if any) replaces the return value
5. Function execution context is destroyed after completion

## Default Parameters

```
function dialogue(user = "Avinash") {  
  console.log(`Hey ${user}...!!`);  
}  
  
dialogue("Avi");    // Hey Avi...!!  
dialogue();         // Hey Avinash...!! (default value used)
```

## Higher-Order Functions (HOF)

A function that:

- **Accepts another function as a parameter**, OR
- **Returns another function**

```
// HOF that accepts a function
function operate(a, b, operation) {
  return operation(a, b);
}
const add = (x, y) => x + y;
const result = operate(5, 3, add); // 8

// HOF that returns a function
function createMultiplier(factor) {
  return function(number) {
    return number * factor;
  };
}
const double = createMultiplier(2);
console.log(double(5)); // 10
```

## Callback Functions (CBF)

A function that is:

- A **callback** is a **function passed as an argument** to another function.
- It is **called (or executed)** after the completion of the function it was passed into.
- Callbacks are a way to **handle asynchronous operations** in JavaScript, especially before **Promises** and **async/await** were introduced.
- They help in **controlling the order of execution** and managing **dependencies** between operations.
- **Passed to another function as a parameter**, OR
- **Returned by another function**

```
// Callback as parameter
function processUser(name, callback) {
  console.log(`Processing ${name}...`);
  callback(name);
}
function welcomeUser(name) {
  console.log(`Welcome ${name}!`);
}
processUser("Avinash", welcomeUser);
// Callback as return value
function getGreeting() {
  return function(name) {
    console.log(`Hello ${name}`);
  };
};
```

```
}  
const greetFunction = getGreeting();  
greetFunction("World");
```

## Function Return Values

Every function in JavaScript returns something:

```
// Explicit return  
function add(a, b) {  
    return a + b; // Returns the sum  
}  
  
// No explicit return  
function greet(name) {  
    console.log(`Hello ${name}`);  
    // Implicitly returns undefined  
}  
  
const sum = add(5, 3); // sum = 8  
const greeting = greet("JS"); // greeting = undefined
```

## Function Scope and Memory Management

```
function parent() {  
    var localVar = "I'm local";  
    function child() {  
        console.log(localVar); // Accesses parent's variable  
    }  
    return child;  
}  
  
const childFunction = parent(); // parent execution context destroyed  
childFunction(); // Still has access to localVar through closure
```

## Advanced Function Concepts

### Function Properties and Methods

Since functions are objects, they have properties:

```
function myFunction() {  
    return "Hello";  
}
```

```
console.log(myFunction.name);    // "myFunction"
console.log(myFunction.length);  // 0 (number of parameters)
// Adding custom properties
myFunction.customProperty = "I'm a custom property";
console.log(myFunction.customProperty);
```

## Function as Constructor

```
function Person(name, age) {
  this.name = name;
  this.age = age;
  this.greet = function() {
    console.log(`Hi, I'm ${this.name}`);
  };
}

const person1 = new Person("Avinash", 25);
person1.greet(); // Hi, I'm Avinash
```

## Best Practices

1. **Use Descriptive Names:** Function names should clearly indicate what they do
2. **Keep Functions Small:** Each function should have a single responsibility
3. **Use Default Parameters:** Provide sensible defaults for optional parameters
4. **Choose Appropriate Function Type:**
  - Use declarations for main functions that might be called before definition
  - Use expressions/arrow functions for callbacks and short operations
  - Use IIFE for initialization code that runs once
5. **Return Values:** Always consider what your function should return
6. **Avoid Global Variables:** Use parameters and return values instead

## Common Pitfalls

1. **Forgetting Return Statement:** Functions return `undefined` if no explicit return
2. **Hoisting Confusion:** Function expressions are not hoisted like declarations
3. **Arrow Function Context:** Arrow functions don't have their own `this`
4. **IIFE Syntax:** Remember both sets of parentheses `(function(){})()`

## Memory Management

- Function execution contexts are created on each call
- Local variables are garbage collected after function completion
- Closures can prevent garbage collection (keep references alive)
- IIFE helps prevent global scope pollution

This comprehensive understanding of functions forms the foundation for advanced JavaScript concepts like closures, async programming, and functional programming patterns.